

Relazione di Laboratorio

Sfruttamento delle Vulnerabilità XSS Reflected e SQL Injection sulla Damn Vulnerable Web Application (DVWA)

Corso: Sicurezza Informatica / Ethical Hacking **Tipologia di attività:** Esercitazione pratica di laboratorio **Ambiente:** Laboratorio virtuale controllato (VirtualBox – Kali Linux / Metasploitable2) **Finalità:** Didattica e accademica

Nota preliminare sull'eticità dell'attività Tutte le attività descritte nella presente relazione sono state condotte esclusivamente all'interno di un ambiente di laboratorio virtuale, isolato e controllato, appositamente predisposto a scopo didattico. Le macchine coinvolte (Kali Linux, in qualità di sistema attaccante, e Metasploitable2, contenente l'applicazione volutamente vulnerabile DVWA) sono strumenti open-source progettati e distribuiti proprio per l'apprendimento delle tecniche di penetration testing in totale sicurezza. Nessuna delle tecniche descritte è stata né sarà applicata a sistemi reali, produttivi o non espressamente autorizzati. Si ricorda, come riportato anche nel banner di avvio della macchina Metasploitable2, che tale sistema non deve mai essere esposto a reti non fidate o pubbliche.

Indice

1. Sintesi
 2. Introduzione
 3. Obiettivi dell'esercitazione
 4. Contesto delle minacce e rilevanza delle vulnerabilità trattate
 5. Ambiente di laboratorio e metodologia
 6. Fase 1 – Configurazione del laboratorio e verifica della connettività
 7. Fase 2 – Accesso alla macchina Metasploitable2 e individuazione della DVWA
 8. Fase 3 – Autenticazione alla DVWA e impostazione del livello di sicurezza
 9. Fase 4 – Sfruttamento della vulnerabilità XSS Reflected
 10. Fase 5 – Sfruttamento della vulnerabilità SQL Injection
 11. Analisi complessiva dei risultati
 12. Strategie di mitigazione e best practice
 13. Considerazioni etiche e legali
 14. Conclusioni e lezioni apprese
 15. Riferimenti bibliografici
-

1. Sintesi

La presente relazione documenta lo svolgimento di un'esercitazione pratica di laboratorio incentrata sull'identificazione e sullo sfruttamento controllato di due classi di vulnerabilità tra le più diffuse nelle applicazioni web: il **Cross-Site Scripting (XSS) di tipo Reflected** e la **SQL Injection non blind**. L'attività è stata condotta utilizzando la piattaforma didattica **Damn Vulnerable Web Application (DVWA)**, ospitata sulla macchina virtuale **Metasploitable2**, attaccata dalla postazione **Kali Linux**. Il lavoro ha previsto la configurazione del laboratorio virtuale, la verifica della raggiungibilità di rete tra le macchine, l'impostazione del livello di sicurezza dell'applicazione a *low* e, infine, lo sfruttamento pratico delle due vulnerabilità sopra citate, con relativa documentazione fotografica di ogni fase. I risultati ottenuti confermano l'efficacia delle tecniche illustrate durante la lezione teorica e permettono di trarre considerazioni utili in merito alle contromisure di difesa applicabili in contesti reali.

2. Introduzione

Le vulnerabilità XSS e SQL Injection figurano, ormai da diversi anni, tra le principali categorie di rischio individuate dall'OWASP (Open Worldwide Application Security Project) per le applicazioni web. Nonostante la loro natura relativamente "classica" rispetto a minacce più recenti legate all'intelligenza artificiale o agli attacchi alla supply chain, tali vulnerabilità continuano a rappresentare un vettore di attacco estremamente diffuso, soprattutto in applicazioni sviluppate senza un'adeguata attenzione alla validazione e alla sanificazione degli input utente.

Al fine di comprendere in maniera pratica il funzionamento, le cause e le possibili contromisure di queste vulnerabilità, è stato predisposto un laboratorio virtuale basato su due macchine: una macchina **Kali Linux**, utilizzata come postazione dell'attaccante (penetration tester), e una macchina **Metasploitable2**, appositamente progettata per contenere applicazioni web volutamente vulnerabili, tra cui la DVWA. Quest'ultima è un progetto open-source ampiamente utilizzato in ambito accademico e formativo per l'addestramento pratico alle tecniche di penetration testing.

3. Obiettivi dell'esercitazione

Secondo quanto indicato nella consegna dell'esercizio, gli obiettivi didattici della presente attività sono stati i seguenti:

- Configurare correttamente l'ambiente virtuale in modo che la macchina DVWA risultasse raggiungibile dalla macchina Kali Linux (attaccante);
- Verificare la comunicazione di rete tra le due macchine tramite il comando **ping**;
- Accedere alla DVWA dal browser della macchina Kali Linux;
- Impostare il livello di sicurezza dell'applicazione su **LOW**, al fine di rendere le vulnerabilità pienamente sfruttabili senza filtri o contromisure applicative;
- Individuare e sfruttare con successo una vulnerabilità di tipo **XSS Reflected**;

- Individuare e sfruttare con successo una vulnerabilità di tipo **SQL Injection** non blind, utilizzando le tecniche presentate durante la lezione teorica.

4. Contesto delle minacce e rilevanza delle vulnerabilità trattate

Prima di procedere alla parte pratica, è utile inquadrare le vulnerabilità applicative in un contesto più ampio. Come evidenziato anche dalla documentazione di riferimento fornita a supporto dell'esercitazione, il panorama delle minacce informatiche risulta oggi caratterizzato da una crescente sofisticazione degli attacchi, resa possibile anche dall'integrazione dell'intelligenza artificiale nelle fasi di ricognizione e social engineering, nonché dalla diffusione di modelli di attacco automatizzati come il *Ransomware-as-a-Service*.

In questo scenario, le vulnerabilità applicative "classiche" come XSS e SQL Injection restano comunque un punto di ingresso privilegiato per gli attaccanti, poiché derivano da errori di programmazione fondamentali — la mancata validazione e sanificazione degli input utente — che, se non corretti, possono compromettere l'integrità dei dati e la sicurezza dell'intera infrastruttura, in linea con quanto sottolineato dal principio "mai fidarsi, verificare sempre" alla base dei moderni modelli di difesa **Zero Trust**. È proprio in ottica di prevenzione che risulta fondamentale, anche per i futuri sviluppatori e amministratori di sistema, comprendere in prima persona le dinamiche di sfruttamento di tali vulnerabilità, così da poterle riconoscere e mitigare efficacemente.

5. Ambiente di laboratorio e metodologia

5.1 Componenti dell'ambiente virtuale

Componente	Ruolo	Sistema operativo	Indirizzo IP
Kali Linux 2026.2	Macchina attaccante (penetration tester)	Debian-based (Kali)	Rete interna VirtualBox
Metasploitable2	Macchina target (vittima), contenente la DVWA	Ubuntu (Linux)	192.168.50.101

Entrambe le macchine sono state eseguite tramite **Oracle VirtualBox**, configurate su una rete interna isolata, in modo da garantire la reciproca visibilità senza esporre le macchine a reti esterne o non fidate, coerentemente con l'avviso di sicurezza mostrato dal banner di avvio della macchina Metasploitable2 (*"Warning: Never expose this VM to an untrusted network!"*).

5.2 Metodologia seguita

L'esercitazione è stata condotta seguendo un approccio metodologico strutturato in fasi sequenziali, tipico delle attività di penetration testing:

1. **Ricognizione e verifica della connettività** (network discovery/connectivity check);
2. **Enumerazione dei servizi disponibili** sulla macchina target;
3. **Accesso all'applicazione target** (DVWA) e configurazione del livello di sicurezza;
4. **Sfruttamento delle vulnerabilità** (exploitation), con relativa raccolta di evidenze;
5. **Analisi dei risultati** e formulazione di considerazioni in merito alle contromisure.

Ogni fase è descritta in dettaglio nei paragrafi successivi, con riferimento alle evidenze fotografiche raccolte durante lo svolgimento pratico dell'esercizio.

6. Fase 1 – Configurazione del laboratorio e verifica della connettività

6.1 Obiettivo

Il primo passo dell'esercitazione ha previsto la configurazione della rete virtuale in modo che la macchina Kali Linux potesse comunicare con la macchina Metasploitable2, e la successiva verifica della connettività tramite il comando **ping**.

6.2 Comandi utilizzati e risultati

Dal terminale della macchina Kali Linux è stato eseguito il seguente comando:

```
ping 192.168.50.101
```

L'output ottenuto ha mostrato una comunicazione corretta tra le due macchine, con **0% di pacchetti persi** su un totale di 3 pacchetti trasmessi e ricevuti, e un tempo di andata e ritorno (RTT) medio pari a circa 0,963 ms (min 0,248 ms – max 2,015 ms).

Figura 1 – Output del comando **ping dalla macchina Kali Linux verso la macchina Metasploitable2 (192.168.50.101), a conferma della corretta connettività di rete.**

6.3 Analisi del risultato

L'esito positivo del comando **ping** ha confermato che le due macchine virtuali erano correttamente collocate sulla medesima rete interna e in grado di scambiarsi pacchetti ICMP, condizione necessaria e propedeutica per procedere con le successive fasi di attacco applicativo.

7. Fase 2 – Accesso alla macchina Metasploitable2 e individuazione della DVWA

7.1 Obiettivo

Verificata la connettività di rete, si è proceduto ad accedere, tramite il browser web presente sulla macchina Kali Linux, alla pagina principale dei servizi esposti dalla macchina Metasploitable2, al fine di individuare l'applicazione target DVWA.

7.2 Risultati ottenuti

Digitando l'indirizzo <http://192.168.50.101> nel browser, è stata visualizzata la pagina di benvenuto di Metasploitable2, contenente il banner ASCII identificativo del sistema, l'avviso relativo al non utilizzo in reti non fidate e l'elenco delle applicazioni web disponibili sulla macchina, tra cui:

- TWiki
- phpMyAdmin
- Mutillidae
- **DVWA**
- WebDAV

Figura 2 – Pagina principale della macchina Metasploitable2, raggiunta dal browser della macchina Kali Linux, con l'elenco dei servizi web disponibili, incluso il collegamento alla DVWA.

7.3 Analisi del risultato

La pagina ha confermato la corretta esposizione dei servizi web sulla macchina target e ha permesso di individuare il collegamento diretto alla Damn Vulnerable Web Application, oggetto della successiva fase di attacco.

8. Fase 3 – Autenticazione alla DVWA e impostazione del livello di sicurezza

8.1 Accesso alla pagina di login

Selezionando il collegamento alla DVWA, il browser è stato reindirizzato alla pagina di autenticazione, disponibile all'indirizzo <http://192.168.50.101/dvwa/login.php>.

Figura 3 – Pagina di login della DVWA, con i campi Username e Password.

L'accesso è stato effettuato utilizzando le credenziali di default previste dall'applicazione stessa ([admin](#) / [password](#)), come indicato nella pagina di login.

8.2 Impostazione del livello di sicurezza

Come richiesto dalla consegna dell'esercizio, è stato necessario impostare il livello di sicurezza dell'applicazione su **LOW**, al fine di disattivare i meccanismi di validazione e sanificazione degli input che, nei livelli superiori (*medium, high*), renderebbero le vulnerabilità più difficili o impossibili da sfruttare direttamente.

L'operazione è stata eseguita tramite la sezione **DVWA Security**, presente nel menù laterale dell'applicazione, selezionando il valore **low** dal menù a tendina e confermando con il pulsante *Submit*.

Figura 4 – Pannello "DVWA Security" con il livello di sicurezza impostato correttamente su low e il modulo PHPIDS (Intrusion Detection System) disattivato, come confermato dal messaggio "Security level set to low".

8.3 Analisi del risultato

L'impostazione del livello di sicurezza su *low* rappresenta una scelta didattica intenzionale, poiché consente di osservare in maniera diretta il comportamento vulnerabile dell'applicazione senza l'interferenza di filtri lato server, replicando così le condizioni tipiche di un'applicazione sviluppata senza adeguate pratiche di *secure coding*.

9. Fase 4 – Sfruttamento della vulnerabilità XSS Reflected

9.1 Descrizione della vulnerabilità

Il **Cross-Site Scripting Reflected** è una vulnerabilità che si verifica quando un'applicazione web restituisce all'interno della risposta HTTP, senza un'adeguata codifica o sanificazione, un input fornito dall'utente stesso all'interno della medesima richiesta. Ciò consente a un attaccante di iniettare codice script (tipicamente JavaScript) che viene eseguito nel contesto del browser della vittima, con conseguenze che possono spaziare dal furto di sessione (*session hijacking*) alla redirectione verso siti malevoli, fino al defacement della pagina.

9.2 Metodologia e comando utilizzato

All'interno della sezione **XSS reflected** della DVWA, è stato individuato un campo di input testuale con etichetta "*What's your name?*", il cui valore veniva restituito senza filtri all'interno della pagina di risposta. È stato quindi inserito il seguente payload:

```
<script>alert('XSS Attack!');</script>
```

Figura 5 – Inserimento del payload XSS nel campo di input della sezione "XSS reflected" della DVWA. Il campo restituisce il valore "Hello" seguito dal contenuto immesso, senza applicare alcuna sanificazione dell'input.

Dopo l'invio del modulo tramite il pulsante *Submit*, il codice script iniettato è stato eseguito direttamente dal browser, generando un pop-up di alert contenente il testo "XSS Attack!".

Figura 6 – Esecuzione del payload XSS lato client: il browser mostra correttamente il pop-up di alert generato dallo script iniettato, a conferma dell'avvenuto sfruttamento della vulnerabilità.

9.3 Analisi del risultato

L'esecuzione del pop-up JavaScript costituisce una prova inequivocabile del fatto che l'input fornito dall'utente non è stato sottoposto ad alcuna procedura di *output encoding* (ad esempio la codifica dei caratteri speciali `<`, `>`, `'`) prima di essere restituito all'interno della pagina HTML. Ciò conferma la presenza della vulnerabilità XSS Reflected nella sezione analizzata dell'applicazione, in condizioni di livello di sicurezza *low*.

In un contesto reale, un payload analogo potrebbe essere utilizzato per rubare i cookie di sessione dell'utente vittima (ad esempio tramite `document.cookie`), effettuare il *defacement* della pagina, oppure reindirizzare l'utente verso un sito di phishing, qualora il link malevolo venisse distribuito tramite email o messaggistica (da qui la definizione "reflected", poiché il payload deve essere veicolato attraverso una richiesta costruita ad hoc dall'attaccante, ad esempio tramite un link).

10. Fase 5 – Sfruttamento della vulnerabilità SQL Injection

10.1 Descrizione della vulnerabilità

La **SQL Injection** è una vulnerabilità che si manifesta quando un'applicazione costruisce dinamicamente query SQL concatenando direttamente l'input fornito dall'utente, senza utilizzare tecniche di parametrizzazione (*prepared statements*) o un'adeguata validazione. Questo consente a un attaccante di alterare la struttura logica della query originale, ottenendo accesso non autorizzato ai dati contenuti nel database, o addirittura la possibilità di modificarli o eliminarli.

10.2 Metodologia e comando utilizzato

Nella sezione **SQL Injection** della DVWA è presente un campo denominato "*User ID*", pensato per restituire nome e cognome di un utente a partire dal relativo identificativo numerico. È stato inserito il seguente payload, classico per l'individuazione di vulnerabilità SQL Injection non blind:

' OR '1'='1' --

Tale input, concatenato alla query SQL originale lato server, ha trasformato la clausola **WHERE** della query in una condizione sempre vera ('1'='1'), facendo sì che la query restituisse **tutti i record** presenti nella tabella degli utenti, anziché il singolo utente corrispondente a un ID specifico. Il doppio trattino (--) è stato utilizzato per commentare la parte residua della query originale, evitando errori di sintassi.

Figura 7 – Risultato dell'iniezione SQL tramite il payload ' OR '1'='1' -- nel campo "User ID": l'applicazione restituisce l'intero elenco degli utenti presenti nel database (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith), anziché un singolo record.

10.3 Analisi del risultato

L'output ottenuto ha mostrato in chiaro i dati di **cinque utenti** presenti nel database dell'applicazione, confermando che la query eseguita lato server non era protetta da meccanismi di parametrizzazione degli input. Questo comportamento dimostra come, tramite un singolo campo di input non adeguatamente validato, sia stato possibile eludere completamente la logica applicativa prevista (restituzione di un singolo utente in base all'ID) ed estrarre l'intero contenuto della tabella.

In un contesto applicativo reale, tecniche analoghe (eventualmente estese con clausole **UNION SELECT**) potrebbero consentire a un attaccante di estrarre informazioni sensibili quali credenziali di accesso, dati personali degli utenti o informazioni riservate dell'organizzazione, con impatti potenzialmente molto gravi sulla riservatezza e sull'integrità dei dati.

11. Analisi complessiva dei risultati

La tabella seguente riassume le vulnerabilità individuate, le tecniche utilizzate e gli esiti ottenuti.

Vulnerabilità	Sezione DVWA	Payload utilizzato	Esito	Evidenza
XSS Reflected	XSS reflected	<code><script>alert('XSS Attack!');</script></code>	Esecuzione di codice JavaScript arbitrario nel browser della vittima	Figure 5–6

SQL Injection (non blind)	SQL Injection	' OR '1'='1' --	Bypass della logica applicativa ed estrazione di tutti i record utente dal database	Figura 7
------------------------------	---------------	-----------------	---	----------

Entrambe le vulnerabilità sono state sfruttate con successo grazie all'assenza di controlli di validazione e sanificazione degli input, resa possibile — e volutamente accentuata a fini didattici — dall'impostazione del livello di sicurezza dell'applicazione su *low*. È interessante osservare come, a fronte di cause tecniche differenti (mancata codifica dell'output nel caso XSS, mancata parametrizzazione delle query nel caso SQLi), entrambe le vulnerabilità condividano la medesima radice concettuale: la **fiducia implicita nell'input fornito dall'utente**, in netto contrasto con il principio *Zero Trust* ("mai fidarsi, verificare sempre") oggi considerato uno standard nella progettazione di sistemi sicuri.

12. Strategie di mitigazione e best practice

Per ciascuna delle vulnerabilità sfruttate, si riportano di seguito le principali contromisure applicabili in un contesto di sviluppo reale.

12.1 Mitigazioni per il Cross-Site Scripting (XSS)

- **Output encoding/escaping**: codificare sistematicamente i caratteri speciali (`<`, `>`, `"`, `'`, `&`) prima di restituire dati generati dall'utente all'interno di pagine HTML;
- **Content Security Policy (CSP)**: definire policy restrittive che impediscano l'esecuzione di script inline o provenienti da fonti non autorizzate;
- **Validazione degli input** lato server, basata su liste di caratteri consentiti (*allow-list*) piuttosto che su liste di caratteri vietati;
- Utilizzo di framework moderni che effettuano l'escaping automatico dei dati in output (ad esempio template engine con auto-escaping attivo).

12.2 Mitigazioni per la SQL Injection

- **Prepared statements / query parametrizzate**: separare in modo netto il codice SQL dai dati forniti dall'utente, impedendo che l'input possa alterare la struttura della query;
- **Utilizzo di ORM (Object-Relational Mapping)** che gestiscono automaticamente la parametrizzazione delle query;
- **Principio del privilegio minimo**: configurare l'utenza del database utilizzata dall'applicazione con i permessi strettamente necessari, evitando account con privilegi amministrativi;
- **Web Application Firewall (WAF)**: come ulteriore livello difensivo, non sostitutivo delle corrette pratiche di sviluppo;
- **Validazione e tipizzazione rigorosa degli input** (ad esempio, verificare che un campo atteso come numerico contenga effettivamente solo cifre).

12.3 Considerazioni generali

Le contromisure sopra descritte si inseriscono in un più ampio framework di difesa "in profondità" (*defense in depth*), che prevede l'adozione di più livelli di controllo indipendenti. A tal proposito, risultano rilevanti anche misure organizzative complementari quali l'autenticazione multi-fattore resistente al phishing (ad esempio tramite chiavi hardware FIDO2 o passkey, in sostituzione dei meno sicuri OTP via SMS), la cifratura dei dati sia *at rest* che *in transit* (ad esempio tramite TLS 1.3), nonché il rispetto dei principali framework normativi applicabili (GDPR, NIS2, DORA per il settore finanziario), che impongono standard minimi di sicurezza e resilienza operativa alle organizzazioni.

13. Considerazioni etiche e legali

Si ribadisce che l'intera attività descritta nella presente relazione è stata condotta esclusivamente in un ambiente di laboratorio virtuale, isolato e privo di connessioni verso reti esterne, utilizzando strumenti (DVWA, Metasploitable2) appositamente progettati e distribuiti dalla comunità open-source a fini didattici. Le tecniche di attacco illustrate non sono state, e non devono in alcun caso essere, applicate a sistemi informatici reali senza una preventiva ed esplicita autorizzazione scritta da parte del legittimo proprietario del sistema (nella forma di un contratto di *penetration testing* o *rules of engagement*), pena la violazione delle normative vigenti in materia di accesso abusivo a sistemi informatici (in Italia, art. 615-ter del Codice Penale).

14. Conclusioni e lezioni apprese

L'esercitazione svolta ha permesso di consolidare, attraverso un approccio pratico, le conoscenze teoriche relative a due delle vulnerabilità web più diffuse e storicamente rilevanti: il Cross-Site Scripting Reflected e la SQL Injection. In particolare, è stato possibile osservare in prima persona:

- l'importanza della fase di ricognizione preliminare (verifica della connettività di rete e individuazione dei servizi esposti);
- come l'assenza di validazione e sanificazione degli input possa essere sfruttata con payload relativamente semplici per compromettere la sicurezza di un'applicazione web;
- la differenza concettuale tra una vulnerabilità che agisce lato client (XSS, con impatto sul browser dell'utente) e una che agisce lato server (SQL Injection, con impatto diretto sul database);
- l'efficacia, anche a scopo dimostrativo, delle contromisure standard oggi raccomandate dalla comunità di sicurezza informatica (OWASP in primis), quali l'output encoding, le query parametrizzate e l'adozione di un modello di sicurezza Zero Trust.

L'attività ha inoltre evidenziato come la sicurezza applicativa non possa prescindere da una progettazione consapevole fin dalle prime fasi dello sviluppo software (*security by design*), e come la formazione pratica su ambienti controllati come quello utilizzato in questo esercizio

rappresenti uno strumento didattico fondamentale per la formazione di futuri professionisti della cybersecurity.